WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Created by the Aerospace Cyber Defence Centre of Excellence (CEC)

# Against KAPE and EZ

# WAC (Windows Artefacts Collector)

## Windows artefact collection software for digital forensic investigation



SYSTEM ARTEFACTS

USER FILES

DIGITAL EVIDENCE

A TOOL IN SUPPORT OF DIGITAL INVESTIGATION

# WAC — Windows Artefacts Collector

Against KAPE and Eric Zimmerman's tools


| | |
|---|---|
| Software version | 3.0.0 |
| Document edition | October 2026 |
| Publisher | Centre d'Excellence Cyberdéfense Aérospatiale (CEC) |
| | École de l'air et de l'espace, air base 701, Salon-de-Provence |



This document confronts WAC with the community's reference tools: KAPE for collection, Eric Zimmerman's command-line tools for decoding. On the same exhibits of the same machine, it compares running times, what each one takes, the accuracy of the values — every difference settled by an independent reference —, the output formats, and the traces each one leaves on the examined machine. It says what the confrontation taught WAC, and what it still holds against it.

# Contents


**1 The method** **2**

1.1 What is compared . . . . . 2

1.2 Who is right when they differ . . . . . 2

1.3 The test bench . . . . . 3

**2 Summary** **4**

**3 Running times** **6**

3.1 The method . . . . . 6

3.2 Collection . . . . . 6

3.3 Conversion and decoding . . . . . 7

3.4 Collection from one same state . . . . . 9

**4 What each one takes** **10**

4.1 The same exhibits, different files . . . . . 10

4.2 What is recorded of each exhibit . . . . . 10

**5 Traces left on the examined machine** **12**

5.1 The measure . . . . . 12

5.2 What each one leaves . . . . . 12

5.3 What the measure fixed in WAC . . . . . 14

**6 Accuracy** **15**

6.1 In favour of WAC . . . . . 15

6.2 In favour of the EZ tools . . . . . 16

**7 Output formats** **17**

7.1 What each tool writes . . . . . 17

7.2 What sets them apart . . . . . 17

**8 What the confrontation taught WAC** **19**

**9 Limits of the comparison** **20**

CHAPTER 1

# The method

## 1.1 What is compared

Two questions, two comparisons.

**Collection: WAC against KAPE**
Both tools collect the same machine one after the other; what each copies is compared, the time it takes, and the SHA-1 of the files both copy.

**Traces left: WAC, KAPE and the EZ tools**
Each one is started from the same state of the same machine, under Sysmon; what it writes there is recorded, what Windows writes there because of it, and what a monitoring tool sees of it (chapter 5).

**Decoding: WAC against the EZ tools**
On the *same exhibits* — the exhibit store of a WAC collection —, WAC converts the whole collection and the matching Eric Zimmerman tool decodes, on a copy, the exhibits of its artefact; both outputs are compared record by record and field by field.

## 1.2 Who is right when they differ

A difference between two tools does not say which one is wrong. Every difference was therefore settled by a third source, independent of both:

- Windows itself: the XML it renders for an event (`Get-WinEvent`), a file's date, the NTFS object identifier (`fsutil objectid`), the live registry, a file's hash (`Get-FileHash`);
- another implementation: `dissect.esedb` (Fox-IT) for ESE databases;
- the format itself: sequence numbers and Marvin32 digests of the hive logs, read byte by byte;
- two artefacts of the same machine cross-checked: Prefetch for execution, a row's timestamp for its other dates.

Each difference is then classed: *defect of WAC* (fixed), *error of the EZ tool* (checked), or *convention* of either — two forms of one value.

## 1.3 The test bench

| | |
|---|---|
| Machine | Windows 11 Pro (build 26200), virtual machine |
| EZ tools | version 2026.5.0 (.NET 9.0.20) |
| KAPE | version 1.3.0.2, target `KapeTriage` |
| WAC | version 3.0.0 with the fixes this confrontation brought |
| Traces measure | clone of the machine, no network, frozen; Sysmon 15.22; tools on a USB key (K:), one run per tool |

Everything can be replayed: `vmtest/ez-compare.sh <artefact>` times the EZ tool on a copy of the exhibits and compares its output with WAC's conversion; `vmtest/lib/timing-table.py` sets the time of each step side by side; `vmtest/kape-compare.sh` does the same for collection; `vmtest/footprint/run-footprint.sh` replays the measure of the traces and `footprint-diff.py` analyses it. The EZ tools ran with their full date precision (`--mp`, or `--dt` with seven decimals where `--mp` is not enough), so as not to blame them for a loss of precision a setting avoids.

CHAPTER 2

# Summary

For each artefact, the outputs of WAC and of the EZ tool are paired record by record, then compared field by field. The last column gives each difference found and who is right, established by an independent source (chapter 6); a field not listed there has the same value on both sides.

| Artefact — tool | Compared | Fields compared | Differences, and who is right |
|---|---|---|---|
| $MFT — MFTECmd | 361,920 records | path, state (deleted, directory), 8 dates, size, MFT entry | sizes of 1,491 compressed files (WOF), 8 open files, 1 deleted file: WAC, as Windows |
| EVTX — EvtxECmd | 211,525 events | identifier, date, provider, machine, event data | 10 values: WAC, as Windows; 435 Perflib events EvtxECmd does not read; cleared logs it ignores |
| Prefetch — PECmd | 478 files | executable, hash, runs and their dates, volumes, loaded files | hash without its leading zero in PECmd (23); two volumes' directories run together (1) |
| Shortcuts — LECmd | 26 files | target, dates to 100 ns, size, volume, machine, arguments | none |
| Jump lists — JLECmd | 60 entries | path, counter, pinning, dates, each entry's shortcut | none |
| Shellbags — SBECmd | 46 folders | name, rank, key and folder dates, MFT entry | wrong NodeSlot at first level in SBECmd (9); "explored" always false |
| Amcache — AmcacheParser | 1,479 entries | the 124 shared columns of the 7 tables | 3,450 values absent from the hive, invented by AmcacheParser; partial SHA-1 not flagged |
| ShimCache — AppCompat-CacheParser | 367 entries | path, date, rank | none; AppCompatCacheParser's misleading "executed" flag is not reported |

| Artefact — tool | Compared | Fields compared | Differences, and who is right |
|---|---|---|---|
| Recycle bin — RBCmd | 66 files | original path, size, deletion date | none |
| SRUM — SrumECmd | 91,008 rows | the 5 tables, all their columns | 16 wrong energy rows in SrumECmd (dissect.esedb); an unclean database it refuses |
| Registry — RECmd | 9 artefacts | UserAssist, BAM, USBSTOR, accounts, Run, MuiCache, typed paths, searches, trusted documents | 4 hives rolled back by RECmd; a deleted account still listed |
| Collection — KAPE | 1,416 files taken by both | SHA-1 fingerprint | 23 different: files Windows rewrote between the two collections |

**Key point**

On everything both sides read, WAC gives the same values as the reference tools, or more accurate ones, each time established by an independent source. The confrontation also revealed 28 defects of WAC, all fixed (chapter 8), and coverage gaps, all filled since (section 6.2).

Started from a key, WAC writes nothing on the examined machine: no file, no registry value, no event in Windows' logs. What remains are the traces Windows leaves of any program started (Amcache, ShimCache), and what a monitoring in place records of its execution: Sysmon, process creation auditing, an EDR. KAPE and the EZ tools, .NET programs, write to the user's registry and, for KAPE's modules, files on the system volume (chapter 5).

CHAPTER 3

# Running times

## 3.1 The method

WAC is not measured artefact by artefact. A conversion run for a single artefact makes it pay, each time, what it really pays once: checking the seal and the fingerprints of the exhibit store, rebuilding the working copy, repairing the hives. A collection run for a single artefact makes it open and walk the volume again each time. The times below therefore come from *one* full collection and *one* full conversion. WAC records the time of each of its steps in `investigation.json` and `conversion.json`. Each artefact is set against the EZ tool that decodes it. The steps the other tools do not have are given apart. `vmtest/lib/timing-table.py` puts them side by side.

## 3.2 Collection

KAPE (target `KapeTriage`) and WAC collect the same machine, one right after the other.

| Tool, setting | Time | Files | Volume |
|---|---|---|---|
| KAPE, `KapeTriage` | 44 s | 1,427 | 0.96 GB |
| WAC, without the executables (`binary: none`) | 41 s | 2,178 | 2.62 GB |
| WAC, kit as shipped (`binary: unverified`) | 488 s | 72,887 recorded | 3.66 GB |

Without the executables, WAC is a little faster than KAPE and brings back 2.7 times as much data. Its steps, measured in this run:

| WAC step | Time | In KAPE |
|---|---|---|
| Checks: Windows version, rights, free space on the medium | 0.00 s | no space check |
| Observation of the running machine: sessions, processes, network, services, clock | 0.16 s | — |
| System hives, raw NTFS reading; the volume is opened once, and its directories cached for the whole collection | 2.09 s | copy, raw reading for locked files |
| User hives | 0.23 s | same |

| WAC step | Time | In KAPE |
|---|---|---|
| Artefact files (logs, Prefetch, shortcuts, browsers…) | 18.10 s | same |
| $MFT, $UsnJrnl, $LogFile, $Boot, $Secure:$SDS | 6.03 s | same |
| Search for FAT and exFAT volumes (USB keys) and reading of their metadata | 0.02 s | — |
| Resource files of the providers the logs cite, for the events' text | 11.15 s | — |
| Sealing: manifest, SHA-256 fingerprint of the manifest | 0.09 s | copy log |
| Outside the steps: start-up, writing of the investigation log | 3.1 s | |

The three fingerprints of each exhibit (MD5, SHA-1, SHA-256) and its NTFS dates are taken during the reading itself: sealing only has to write the manifest. The steps of WAC's own — checks, observation of the machine, FAT volumes, providers' resources, sealing — cost 11.4 s, 11.2 s of them for the providers' resources. They are what lets the analysis workstation render every event's text as Windows displays it.

With the executables (the shipped kit's setting), the collection takes 488 s, 389 s of them to read every executable, library, driver and script of the volume. WAC checks their Authenticode signature, in memory, against its trust set, and copies only those whose signature does not hold. The 54,467 files read divide as follows:

- 28,180 authenticated as Microsoft and 328 cleared by a third-party chain: fingerprints recorded, files not copied;
- 12,691 Windows differential files (MSDELTA), which are no executables: fingerprints recorded, not copied;
- 2,599 whose signature does not hold, copied (771 MB);
- 10,669 whose content is identical to an exhibit already copied: recorded, pointing to that exhibit, not copied again.

The 43 s of the process observation come, in this setting, from the fingerprints of every loaded module, and the 5 s of the trust set from its loading. KAPE makes none of these checks: it copies its targets' files without examining them.

## 3.3 Conversion and decoding

WAC's conversion reads the exhibit store of a collection, on the analysis workstation. Each EZ tool reads a copy of the same exhibits: the tool is timed alone, the copy is not. That copy is no formality. Run on the exhibit store itself, SrumECmd wrote into it (its `JetInit` lays a checkpoint and logs beside the database), and WAC then refused to convert a store that no longer matched its manifest.

| Artefact | wac | wac --binary | EZ tool | Tool |
|---|---|---|---|---|
| $MFT (C: and G:) | 4.14 s | 4.02 s | 9.2 s | MFTECmd (C: only) |
| Event logs | 10.65 s | 9.97 s | 24.6 s | EvtxECmd |
| Prefetch (478) | 0.33 s | 11.66 s | 2.4 s | PECmd |
| Shortcuts | 0.03 s | 0.04 s | 1.6 s | LECmd |
| Jump lists | 0.03 s | 0.04 s | 0.6 s | JLECmd |
| Shellbags | 0.01 s | 0.01 s | 0.9 s | SBECmd |
| ShimCache | 0.01 s | 0.39 s | 1.2 s | AppCompatCache-Parser |
| Amcache | 0.07 s | 8.96 s | 0.7 s | AmcacheParser |
| Recycle bin | 0.07 s | 0.01 s | 0.5 s | RBCmd |
| SRUM | 1.77 s | 1.72 s | 2.5 s | SrumECmd |
| Registry (Kroll batch) | 0.26 s | 1.43 s | 10.2 s | RECmd |
| **Sum** | 17.4 s | 38.3 s | 54.4 s | the 11 tools |

To these steps add those wac pays once for all the artefacts, and which the EZ tools do not have:

| Shared wac step | Time |
|---|---|
| Check of the seal and of each exhibit's fingerprints | 10.9 s |
| Rebuilding of the working copy from the exhibit store (2.1 GB), each copy checked by its fingerprint | 45.1 s |
| Repair of the working copy's hives (replay of their logs) | 0.5 s |
| The suspect's time zone, code page, profiles and account names | 0.0 s |
| **Total** | 56.5 s |

The whole conversion takes 80 s: the 56 s shared, the 17.4 s of the eleven compared artefacts, and 6.4 s for the 38 others, which the EZ tools do not read (change journal, scheduled tasks, thumbnail caches, RDP connections, mail, browsers…). The eleven EZ tools take 54 s together, without a working copy or a check of the exhibit store.

- Once the working copy is made, each wac decoding is faster than the matching EZ tool: the hives are read once, for every registry artefact.
- The heaviest step is the working copy. It is the condition of an analysis that never touches the exhibit store, and the EZ tools need the same copy, as SrumECmd showed.
- With `--binary`, wac fingerprints every file the artefacts cite: +11.3 s for the Prefetch (the loaded libraries), +8.9 s for the Amcache. The EZ tools do not.

## 3.4 Collection from one same state

Measures of the traces comparison (chapter 5): every tool starts from the same frozen state of the machine, started from the key; the duration is that of the task that starts it, in brackets the one the tool reports.

| Tool, setting | Duration | Taken | On the key |
|---|---|---|---|
| KAPE, `!SANS_Triage` | 127 s (115 s) | 2,291 files | 2,286 files, 1.25 GB |
| KAPE, `!SANS_Triage` and `!EZParser` modules | 235 s (223 s) | 2,288 files, 14 tools | 2,345 files, 1.63 GB |
| EZ tools on the live machine (6 tools) | 132 s | $MFT, logs, hives, Prefetch, Amcache, ShimCache | 36 CSV, 0.55 GB |
| WAC, collection (default setting) | 592 s (588 s) | 11,830 exhibits, 67,882 recorded | 8,240 files, 4.9 GB |

WAC's collection is the longest because it reads *every* executable of the machine: the conversion will happen elsewhere, and it does not know yet which ones the artefacts will cite. It computes their fingerprints, checks their signature and copies only those whose signature does not hold; hence 67,882 exhibits recorded for 11,830 copied. The conversion, on the analysis workstation, reads nothing of the machine (chapter 5). KAPE copies the files of its targets without examining them.

CHAPTER 4

# What each one takes

## 4.1 The same exhibits, different files

Latest comparison: KAPE with its `KapeTriage` target, and WAC without the executables (`binary: none`), one right after the other; the files are paired by their original path (WAC's manifest, KAPE's copy log). KAPE takes 1,421 files, WAC 2,206; 1,416 are taken by both.

KAPE takes only 5 files WAC does not: `desktop.ini` files, which only set a folder's icon and display name. WAC takes 790 more: the resource files of the event providers (286), which give the events' text; Defender (123); the shortcuts outside Recent (76); Thunderbird's mail, Explorer's thumbnail caches and the Windows search index (183 in all); the error reports (32); other hives and NTFS metadata; BITS, WMI, DPAPI, hosts, setupapi, the FAT volumes, the running machine's state.

Of the 1,416 shared files, 1,393 have the same SHA-1 fingerprint. The other 23 are files Windows keeps rewriting — $MFT, change journal, Prefetch, Defender's logs, SRUM — and changed between the two collections.

## 4.2 What is recorded of each exhibit

| | **WAC (manifest)** | **KAPE (copy log)** |
|---|---|---|
| Fingerprints | MD5, SHA-1, SHA-256 | SHA-1 |
| File dates | the 4 of `$STANDARD_INFORMATION` and the 4 of `$FILE_NAME`, to the 100 ns, in UTC and in the suspect's local time | 3 of `$STANDARD_INFORMATION` (creation, modification, access), to the 100 ns, in UTC, without the MFT record change |
| Location | MFT record number | path alone |
| Method | stated for every exhibit (raw NTFS reading, live observation) | ordinary copy; "deferred" raw reading for locked files (181 out of 2,292) |
| Failure | code and cause in the manifest, exhibit by exhibit | console and log |
| Integrity | sealed manifest (SHA-256), checked again at every conversion | — |

> **Key point**
>
> The `$FILE_NAME` dates are those an intruder does not rewrite with the ordinary functions: a backdated file (“timestomping”) keeps its real creation date in `$FILE_NAME`. KAPE’s log does not have them; WAC’s manifest gives them for every exhibit, with the MFT record that finds it again in `mft.json`.

CHAPTER 5

# Traces left on the examined machine

## 5.1 The measure

A clone of the test machine, without network, is frozen with Sysmon installed; the tools are on a USB key, drive K:. Every run starts from that same state: the system disk and the key are fresh layers over the frozen images. Once started up, each tool is launched the same way — a scheduled task of the user, with its elevated rights, as an examiner from an elevated prompt.

**Files**
The USN of C:'s change journal is read just before and just after the tool: every file created, changed, renamed or deleted between the two is in it, even one created then deleted during the run. The readings are written to the key, after the end bound.

**Author**
Sysmon gives the tool's process tree and what each process wrote, files and registry. A write of another process that names the tool (Amcache, Prefetch, BAM) is Windows reacting to its start.

**Control**
A run that starts nothing (`ver`) measures the launcher and Windows' own activity; what it shows too is not laid on the tool.

**Deferred**
The ShimCache is written at shutdown only: every run ends with a clean shutdown, a restart and its reading.

## 5.2 What each one leaves

| Tool | Files written on C: | Registry written | Windows' reactions |
|---|---|---|---|
| WAC (collection) | none | none | Amcache: one entry during the run; ShimCache: one entry at shutdown |

| Tool | Files written on C: | Registry written | Windows' reactions |
|---|---|---|---|
| KAPE, targets | none | 4 Internet Settings `ZoneMap` values in the user's hive; 169 open-or-create calls (certificate stores, policies) | Amcache and ShimCache for `kape.exe` |
| KAPE, targets and modules | 28: 14 in `ProgramData\Microsoft\NetFramework\BreadcrumbStore`, 14 in the user's Temp (`system-commandline-sentinel-files`) | the same 4 values, rewritten by 14 of the 15 processes (136 writes); about 2,600 open-or-create calls | Amcache and ShimCache for KAPE and 14 tools |
| EZ tools, live | none | no value; 54 open-or-create calls (policies, `Tcpip`, `Internet Settings`) | Amcache and ShimCache for the 6 tools |

The tools' processes made Windows write no event in its own logs (System, Security, Application, operational logs): those of the measure window are the services' logons, Defender, group policy, common to the control run or unrelated to the tool. Process creation auditing is not active on the clone; active, it would have logged a 4688 and a 4689 per process (chapter 9).

The installed monitoring, though, records each tool's execution: Sysmon's log received, for the tool's processes only,

| Tool | Processes | Raw reads | Files created | Registry |
|---|---|---|---|---|
| WAC | 1 | 2 | 16,382, all on the key; 5,588 deleted on the key | none |
| KAPE, targets | 1 | 2 | 2,553, all on the key | 169 keys opened for creation, 8 values |
| KAPE, targets and modules | 17 | 19 | 2,622 on the key, 27 on C: | 2,611 keys, 179 values |
| EZ tools, live | 6 | 78 | 45, all on the key | 54 keys |

plus the libraries each process loaded (24 for WAC). An EDR draws the same telemetry, and

may report the raw reading of the volumes (`RawAccessRead`).

**Open-or-create**
`RegCreateKeyEx` creates the key if it is missing and opens it otherwise; Sysmon does not tell the two apart. The .NET libraries open the certificate stores and the policy keys that way. A created key changes date; a key merely opened does not.

**The `ZoneMap` values**
Written by WinINet's start-up in every .NET process that loads it, they change the examined user's `NTUSER.DAT` hive, hence its log and the key's date.

**The .NET files**
The runtime leaves one "breadcrumb" per assembly loaded in `BreadcrumbStore`, and the System.CommandLine library a file per tool in the user's Temp. Started alone, the EZ tools left none; started by KAPE, they did.

**Prefetch and BAM**
None of the 22 programs started from the USB key got a Prefetch file or a BAM value, while `cmd.exe`, started from the system disk by the same task, has its BAM value. A tool copied onto the machine and started from there would get them.

**The .NET prerequisite**
KAPE and the EZ tools need the .NET 9 runtime, present on the clone. On a machine without it, installing it writes hundreds of files and keys; WAC is a self-contained executable.

## 5.3 What the measure fixed in WAC

WAC cleared the screen at start-up with `system("cls")`, which starts `cmd.exe`: a process created on the examined machine and the Prefetch of `CMD.EXE` updated, at every collection. The screen is now cleared through the console API, WAC imports no function able to start a process any more, and the build script refuses a version that would import one.

> **Key point**
> Every trace of WAC is explained: it starts no process, writes no file on the machine, no registry value, no event in Windows' logs. What remains is what Windows leaves of any program started — an Amcache entry, a ShimCache entry at shutdown —, and what the monitoring in place records of its execution (Sysmon, process creation auditing, EDR): unavoidable, and described in the harmlessness file.

CHAPTER 6

# Accuracy

## 6.1 In favour of WAC

**Hive logs replayed in order**
The `.LOG1` and `.LOG2` logs keep, behind the recent entries, leftovers of an earlier generation. RECmd applies them in file order, hence after the newer ones. It thus rolls SYSTEM, SECURITY, DEFAULT and a service profile's hive back to an older state: in SYSTEM, sequence 0x3618 instead of 0x3620, and four last BAM executions shown 13 minutes early. WAC sorts the entries by sequence and stops at the break in the chain. The logs themselves confirm it, Marvin32 digests included.

**Events of cleared logs**
A cleared log keeps its records: WAC reads them again and flags them `EvtRecovered`, never presenting them as live; EvtxECmd does not see them.

**Event values as Windows renders them**
Hexadecimal `0x3e7`, GUIDs in braces, booleans `true`, binary `69060000`, dates with their `Z`, U+200E marks kept: the forms Event Viewer shows and one searches for. EvtxECmd changes eight of them, and is wrong on ten values, such as a boolean `False` that Windows renders `true`; each one is settled by the XML Windows renders (`Get-WinEvent`) from the same log.

**Nothing invented**
A value absent from the source is not emitted. AmcacheParser writes 3,450 default values absent from the hive (`IsPeFile: False` for a PE executable), SrumECmd 10,936 (0 or $-1$).

**Partial Amcache SHA-1 flagged**
For a file larger than 30 MiB, Amcache hashes only the first 31,457,280 bytes (measured on `mrt.exe`). WAC says so (`AmcacheSha1CoversBytes`); without it, comparing with the whole file's SHA-1 would wrongly conclude to a modification.

**No misleading field**
The ShimCache "executed" flag, which AppCompatCacheParser gives, says "no" for 105 of the 117 binaries that Prefetch proves were run. WAC does not collect it: that "no" would lead to conclude a program never ran.

**Precision to 100 ns**
WAC gives every date at its source's precision. By default, PECmd, LECmd and SrumECmd round to the second, and LECmd ignores `--mp` in its CSV.

**Dated local time**
Every date is given in UTC and in the suspect's local time, with the offset in force at that date. The EZ tools give UTC only, without saying so by a `Z`.

**Others' readings checked**
SBECmd's "FirstInteracted" and "LastInteracted", RECmd's "CreatedOn" of an account, are readings of key write times. WAC publishes those times themselves: anyone can draw their own reading from them, without inheriting a ready-made one. SBECmd also gives a wrong `NodeSlot` (0) to first-level folders.

**SrumECmd's ESE reading errors**
16 energy rows out of 1,370 carry the content of another row or an identifier absent from the table; dissect.esedb sides with WAC.

**ESE database not closed cleanly**
A SRUM database taken from a running machine is not closed cleanly. SrumECmd stops on it; WAC replays its logs on the working copy (`esent.dll`), then reads it.

**Deleted accounts**
The bench deletes an account (`wacrdp`) after using it. RECmd still lists it, from a SAM rolled back to an earlier state; WAC does not give it, and neither does `Get-LocalUser`.

**Collection**
Each WAC exhibit carries MD5, SHA-1 and SHA-256, the eight NTFS dates of its original and its MFT entry, in a sealed manifest. KAPE gives one SHA-1 and three dates.

## 6.2 In favour of the EZ tools

**Coverage**
No gap is left open. The keys of the Kroll batch WAC ignored are now read: installed programs, Defender exclusions, RecentDocs, MountPoints2, known networks, the event log configuration, among others. Those that trace no activity are left out, with their reason. The files KapeTriage alone took are collected and converted: Chromium browsers, WebCache and INetCache, the tabs open or closed, the favicons, the address bar's predictor, the Windows timeline, shortcuts outside Recent, the Default profile's hive, `$Secure:$SDS`, `$Boot`, Defender's WPP traces and the DPAPI master keys. Only the `desktop.ini` files are left out, of no investigative value.

**Speed on small artefacts**
Taken one by one, Prefetch, Amcache and shortcuts decode faster with the EZ tool: see chapter 3.

**Ready-to-read interpretations**
The EZ tools split and name some fields: executable, date and description of a SRUM application, known folder names, a file's extension. WAC gives the raw value, as Windows stores it.

> **Limit**
> What KAPE takes and WAC does not is mostly application files in the profiles (chapter 4). They match no artefact WAC decodes: their worth depends on the investigation.

CHAPTER 7

# Output formats

## 7.1 What each tool writes

| Tool | Formats | Note |
|---|---|---|
| AmcacheParser | CSV | 8 files, one per table |
| AppCompatCacheParser | CSV | |
| RBCmd | CSV | |
| SrumECmd | CSV | 7 files |
| EvtxECmd | CSV, JSON, XML | |
| JLECmd | CSV, JSON, HTML | |
| LECmd | CSV, JSON, HTML, XML | |
| MFTECmd | CSV, JSON, bodyfile | |
| PECmd | CSV, JSON, HTML | |
| RECmd | CSV, JSON | plus one CSV per plugin |
| SBECmd | CSV, JSON | |
| KAPE | copy of the files, CSV copy log | |
| WAC | JSON for every artefact | one file per artefact, plus the sealed manifest |

## 7.2 What sets them apart

Four tools out of eleven write CSV only. In the others, the JSON transcribes the CSV line: one text line per record (JSON Lines), flat.

**Types**
In the EZ tools, numbers are strings: a run count is written `"RunCount":"458"`, a record number `"EventRecordId":"21572"`. WAC writes numbers, booleans, strings.

**Structures**
In the EZ tools, a list becomes a comma-joined string (the files a Prefetch loaded, its directories), and an event's data are JSON escaped *inside* a string (`Payload`), to decode a second time. WAC nests: a Prefetch's volumes and their directories, a shortcut's items, an event's `Name`/`Value` pairs.

**Dates**
In the EZ tools, UTC is not flagged (`2026-09-30 23:04:46`) and precision depends on an option. WAC gives full ISO 8601, in UTC (…`Z`) and in local time with its offset (…`+02:00`).

**Missing value**
The EZ tools write a default value, or an empty cell. WAC omits the field; an artefact not collected is declared so (`NotRequested`, `NotCollected`), so that an empty list is not taken for an absence of traces.

**Provenance**
WAC's manifest ties every value to the exhibit it comes from, with its hashes and original dates.

**Console**
The EZ tools detail their work on the console; some write the decoding itself there when no file is asked for. WAC writes only its progress there: everything it finds is in the files.

> **Key point**
> For an analyst, the difference shows at the first processing: a WAC JSON loads as it is into an analysis tool or into WAE, with nothing to split again; an EZ output is handled like a spreadsheet, whose numbers must be converted back, lists split again and event data decoded again.

CHAPTER 8

# What the confrontation taught WAC

Each defect below produced a valid JSON that was wrong, or incomplete. All are fixed, with a check that would have caught them.

| Artefact | Defect fixed |
|---|---|
| $MFT | stale extension records merged with the base |
| EVTX | floats cut to 6 digits; binary as “69 06 00 00”; hexadecimal integers and GUIDs in the wrong form; ProcessingErrorData ignored (21 events without data); events of cleared logs not read |
| Shortcuts | URI glued to the block after it; reading 2 bytes off without an ID list; ExtraData blocks not read (machine, tracking); target size not read; null DROIDs emitted |
| Jump lists | open count and MRU rank not read; DestList format version 1 not supported; application name corrupted |
| Shellbags | folder lost with everything below it; the folder’s key time, rank and NodeSlot missing; “Home” folder without a name; the `NTUSER.DAT` tree not read (only `UsrClass.dat`’s was) |
| Amcache | 2 tables out of 14 and 4 to 6 values read: no SHA-1, size, driver or device; a SHA-1 named `Sha1`, read as the whole file’s, renamed `AmcacheSha1` |
| ShimCache | tabs of packaged applications replaced by spaces |
| SRUM | FILETIME dates left as numbers; UTF-16 text read as ANSI |
| Known identifiers | 105 DLL paths of the GUID table (253 occurrences) whose generation had taken `\n` or `\r` for a line break: `system32\ netcorehc.dll` became `system32 etcorehc.dll` |
| Registry | BAM limited to accounts with a profile (a deleted account lost its executions); UserAssist foreground time not read |
| Collection | exhibits whose path passes 260 characters once in the store not written (482 on the full kit): now written in the extended form |
| Start-up | `system("cls")` started `cmd.exe` on the examined machine (traces measure) |
| Test harness | a crashing check silently stopped every following one; the EZ tools ran on the sealed exhibit store itself — SrumECmd wrote into it and WAC refused the next conversion; they now read a copy |

CHAPTER 9

# Limits of the comparison

- One machine, virtual, running Windows 11: some artefacts are few there (2 USB keys, one Explorer search). The formats are the same on a real machine, but not every rare case is represented.
- The two collections follow each other instead of being simultaneous: the files written between them ($MFT, logs, Prefetch) rightly differ.
- The EZ tools ran with their maximum precision options; by default, they lose more.
- The times come from a virtual machine and its disk: they compare the tools with each other, they do not predict the time on a real machine, where the number of executables and the disk's speed weigh most.
- The traces are measured on a clone without network, one run per tool: a write of Windows the control run did not show is not laid on the tool unless Sysmon gives it to one of its processes; it stays listed as not attributed in `footprint-diff.py`'s results.
- Process creation auditing (Security 4688) is not active on the test machine: where it is, every tool leaves an event per process — one for WAC, fifteen for KAPE and its modules.
- The comparison covers the common artefacts. What only one tool reads has no reference here: its accuracy rests on the checks of the validation report.